

Chapter 6: Java Framework

- The Java programming language was originally created in 1995 by James Gosling from Sun Microsystems (currently a subsidiary of Oracle Corporation).
- The goal was to provide a simpler and platform-independent alternative to C++.
- Java is a general-purpose programming language that's used in all industries for almost any type of application.

The Life Cycle of a Java Program

- Java requires the source code of our program to be compiled first.
- It gets converted to either machine-specific code or a bytecode that is understood by some run-time engine or a virtual machine.
- Not only will the program be checked for syntax errors by a Java compiler, but also some other libraries of Java code can be added (linked) to our program after the compilation is complete (deployment stage).
- Technically we can write the source code of our Java program in any plain text editor that we prefer (Notepad, TextEdit, vi, etc.), but to compile our program we'll need additional tools and code libraries that are included in the Java Development Kit (JDK).

Java Runtime Environment (JRE)

Java Runtime Environment (JRE) contains JVM, class libraries, and other supporting files. It does not contain any development tools such as compiler, debugger, etc. Actually JVM runs the program, and it uses the class libraries, and other supporting files provided in JRE. If we want to run any java program, we need to have JRE installed in the system.

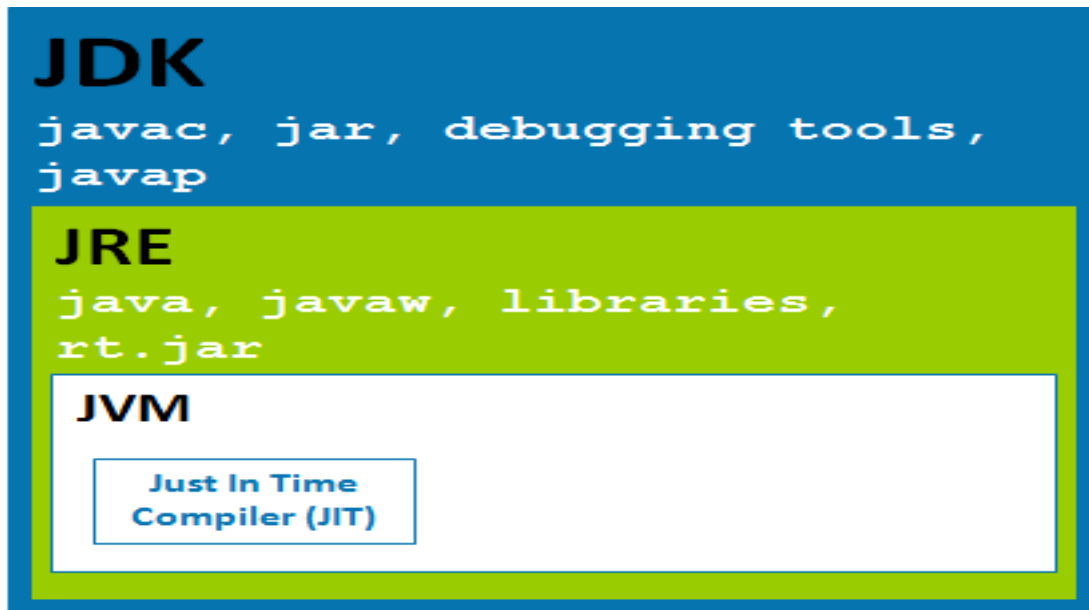
Java Development Kit (JDK)

Java Developer Kit contains tools needed to develop the Java programs, and JRE to run the programs. The tools include compiler (javac.exe), Java application launcher (java.exe), Appletviewer, etc. Compiler converts java code into *byte code*. Java application launcher opens a JRE, loads the class, and invokes its main method. We need JDK, if at all we want to write our own programs, and to compile the program. For running java programs, JRE is sufficient. JRE is targeted for execution of Java files i.e. *JRE = JVM + Java Packages Classes (like util, math, lang, awt, swing etc) + runtime libraries*. JDK is mainly targeted for java development. i.e. we can create a Java file (with the help of Java packages), compile a Java file and run a java file.

Java Virtual Machine (JVM)

The Java Virtual Machine provides a platform-independent way of executing code; programmers can concentrate on writing software, without having to be concerned with how or where it will run. But, note that JVM itself not a platform independent. It only helps Java to be executed on the platform-independent way. When JVM has to interpret the byte codes to machine language, then it has to use some native or operating system specific language to interact with the system. One has to be very clear on platform independent concept. Even there are many JVMs written on Java, however they too have little bit of code specific to the operating systems.

As we all aware when we compile a Java file, output is not an 'exe' but it's a '.class' file. '.class' file consists of Java byte codes which are understandable by JVM. Java Virtual Machine interprets the byte code into the machine code depending upon the underlying operating system and hardware combination. It is responsible for all the things like garbage collection, array bounds checking, etc. JVM is platform dependent.



The JVM is called “virtual” because it provides a machine interface that does not depend on the underlying operating system and machine hardware architecture. This independence from hardware and operating system is a cornerstone of the *write-once run-anywhere* value of Java programs. There are different JVM implementations are there. These may differ in things like performance, reliability, speed, etc. These implementations will differ in those areas where Java specification doesn’t mention how to implement the features, like how the garbage collection process works is JVM dependent, Java spec doesn’t define any specific way to do this.

Difference between JRE, JDK and JVM

In short here are few differences between JRE, JDK and JVM:

- 1) JRE and JDK come as installer while JVM are bundled with them.
- 2) JRE only contain environment to execute java program but doesn’t contain other tool for compiling java program.
- 3) JVM comes along with both JDK and JRE and created when we execute Java program by giving “java” command.

Just in Time Compiler (JIT)

Initially Java has been accused of poor performance because it’s both compiles and interprets instruction. Since compilation or Java file to class file is independent of execution of Java program. Here compilation word is used for byte code to machine instruction translation. JIT are advanced part of Java Virtual machine which optimize byte code to machine instruction conversion part by compiling similar byte codes at same time and thus reducing overall execution time. JIT is part of Java Virtual Machine and also performs several other optimizations such as in-lining function.